

Received April 27, 2022, accepted May 14, 2022, date of publication May 18, 2022, date of current version May 24, 2022.

Digital Object Identifier 10.1109/ACCESS.2022.3176086

Advanced Crowdsourced Test Report Prioritization Based on Adaptive Strategy

PENGHUA ZHU¹, YING LI¹, TONGYU LI², HUIMIN REN³, AND XIAOLEI SUN⁴

¹North China Institute of Aerospace Engineering, Langfang 065099, China

²State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210093, China

³Beijing Aerospace Automatic Control Institute, Beijing 100854, China

⁴Wuhu Institute of Technology, Wuhu 241002, China

Corresponding author: Huimin Ren (appletree_rhm@163.com)

This work was supported in part by the Special Fund for Military-Civilian Integration Development of Hebei Province under Grant JMYB-2020-01, and in part by the Key Project of Natural Science Research in Anhui Higher Education Institutions under Grant KJ2019ZD67.

ABSTRACT Crowdsourced testing is an emerging trend in software testing, which takes advantage of the efficiency of crowdsourced and cloud platforms. Crowdsourced testing has gradually been applied in many fields. In crowdsourced software testing, after the crowdsourced workers complete the test tasks, they submit the test results in test reports. Therefore, in crowdsourced software testing, checking a large number of test reports is an arduous but unavoidable software maintenance task. Crowdsourced test reports are numerous, complex, and need to be sorted to improve inspection efficiency. There are no systematic methods for prioritizing reports in crowdsourcing test report prioritization. However, in regression testing, test case prioritization technology has matured. Therefore, we migrate the test case prioritization method to crowdsourced test report prioritization and evaluate the effectiveness of these methods. We use natural language processing technology and word segmentation to process the text in the test reports. Then we use four methods to prioritize the reports: total greedy algorithm, additional greedy algorithm, genetic algorithm, and ART. The results show that these methods all perform well in prioritizing crowdsourced test reports, with an average APFD of more than 0.8.

INDEX TERMS Crowdsourced software testing, test report prioritization, text classification.

I. INTRODUCTION

Crowdsourcing technology has been widely favored in the field of software engineering research [1] in recent years. In crowdsourcing software testing, the main participants include task requesters, crowdsourcing workers, and crowdsourcing platforms [2]. Crowdsourced works are required to perform test tasks and submit a test report of the system's behavior. The test reports are usually composed of natural language descriptions, sometimes with screenshots. Since crowdsourced testing usually involves many users, the number of test reports can be great, and the task of reviewing these test reports is time-consuming and expensive. Therefore, developers are trying to find methods to prioritize and filter valuable reports.

Previous research has produced test case prioritization techniques. The test case prioritization is to arrange

the execution order of the test cases in the regression test suite to execute them in the order of earlier faults. The existing prioritization algorithms can be divided into the following groups, i.e., greedy algorithm, search-based algorithm, information-retrieval-based algorithm, integrate-linear-programming-based algorithm, and machine-learning-based algorithm [3]. In the field of crowdsourced test report prioritization, there is no mature technology proposed, and the test case prioritization technology has matured. This motivates us to migrate and apply test case prioritization technology to the prioritization of crowdsourced test reports.

In this paper, we apply four test case prioritization techniques to crowdsourced test report prioritization and evaluate the effectiveness of these methods. The four methods are total greedy algorithm, additional greedy algorithm, genetic algorithm, and ART [4]. We collect a total of 723 crowdsourced test reports for five different types of applications. After obtaining the crowdsourced test reports, we use natural language processing (NLP) technology and word segmentation

The associate editor coordinating the review of this manuscript and approving it for publication was Poki Chen ¹.

to process the texts in the test report. First, we put together all the texts of each report. Then, Jieba library¹ is used to perform word segmentation processing on the summarized texts. After word segmentation, we filter out the stop words based on the stop word list.² Finally, we use a one-hot encoded vector to represent each report. After getting the vector, we use the above four algorithms to sort the test report. For the ART method, in addition to the word segmentation method, we also use the Word2Vec model [5], and TF-IDF [6] to encode the texts. The average percentage of faults detected (APFD) [7] is introduced to compare four test report prioritization methods. The experimental results show that four methods perform well in the crowdsourced test report prioritization, and the average APFD is above 0.8.

In this paper, our main contributions are as follows:

- 1) We first apply the test case prioritization method to the prioritization of crowdsourced test reports, adopt corresponding text encoding strategies for different methods, and explore the feasibility of these methods.
- 2) We employ a diverse strategy to rank test reports, and these strategies are evaluated using five crowdsourced test report datasets.

The rest of this paper is structured as follows. In the section II, we introduce the relevant background and motivation. Section III describes four prioritization methods. In section IV and V, we conduct empirical experiments to evaluate the effectiveness of the prioritization of crowdsourced test reports. The related work is reviewed in Section VI. Finally, in section VII, we conclude this paper.

II. BACKGROUND AND MOTIVATION

In this section, we describe the background of crowdsourced software testing to motivate the prioritization of test reports. We also show some sample test reports. Prioritization of test cases is also introduced in this section.

A. CROWDSOURCED SOFTWARE TESTING

In 2006, Howe first proposed the concept of crowdsourcing [8]. Subsequently, many scholars defined crowdsourcing from different angles [9]. In crowdsourcing software engineering, researchers have proposed a large number of related application technologies and application scenarios. Zhang *et al.* derived the definition of crowdsourcing software testing from the definition of crowdsourcing software engineering [10]. Figure 1 shows the procedure of crowdsourcing testing. Testers prepare the software under test and test tasks. Test tasks are divided into sub-tasks. These tasks are released on a crowdsourced platform, and workers bid for test tasks. When a bug is detected, the worker submits a test report online.

In crowdsourced software testing, workers will submit a test report after completing the task. Because crowdsourced

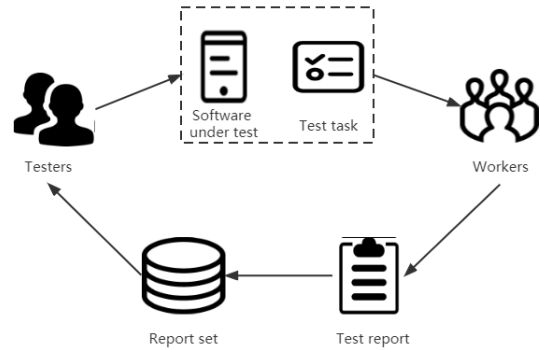


FIGURE 1. The procedure of crowdsourcing testing.

testing involves many tasks, workers usually submit thousands of test reports. Testers manually check all test reports to judge the performance of workers, which is a time-consuming and tedious process. Therefore, this motivates us to prioritize crowdsourced test reports.

In Table 1, we show three examples of crowdsourced test reports. Each test report is mainly composed of Environment, Input, Expect Output, Actual Output, and Screenshot. Environment refers to the test environment, including hardware and software configuration. Input is test data and operation steps. Workers follow the corresponding test steps to carry out the test. Expect Output is the result that worker expects the software to run. Actual Output is the result of the software actually running. Screenshot is the state of the software when a bug occurs.

B. TEST CASE PRIORITIZATION

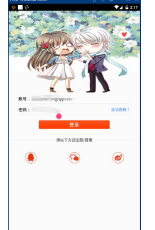
Prioritization of test cases is to prioritize test cases that need to be re-executed in regression testing. Test cases are executed in the prioritized order to detect more faults earlier. The test case prioritization can be defined as the following process: given any test suite T , the test case prioritization is to find a permutation T' of T that satisfies $f(T') \geq f(PT)$, where PT represents a set of possible permutations of T and f is a function from PT to the real number, when applied to an ordering, an evaluation of the ordering occurs [11].

Test case prioritization algorithms can be divided into several groups, i.e., greedy algorithm, search-based algorithm, information-retrieval-based algorithm, integrate-linear-programming-based algorithm, and machine-learning-based algorithm [3]. The greedy algorithm is widely used to solve the problem of test case prioritization, and it searches for the optimal local solution of prioritization. Since the greedy algorithm does not always get the optimal solution in the solution space, search-based algorithms are used to solve the prioritization problem. Integrate-linear programming-based algorithm transforms the prioritization of test cases into a formula construction and solution process.

¹<https://github.com/fxsjy/jieba>

²<https://github.com/goto456/stopwords>

TABLE 1. Examples of test reports.

No.	Environment	Input	Description	Screenshot
Report1	Operating System: Android 4.3 Phone model: Vivo X3L Screen Resolution: 1920 × 1080 System Language: Chinese	1. Click the "My" button 2. Click the settings icon in the upper right corner 3. Click "Personal Information" 4. Click "Date of Birth" to select 2017.10.21	Personal information can be stored normally.	
Report2	Operating System: Android 4.3 Phone model: Vivo X3L Screen Resolution: 1920 × 1080 System Language: Chinese	1. Disconnect from the network 2. Use email to log in 3. Click to log in	Prompt "the network is not connected".	
Report3	Operating System: Android 4.3 Phone model: Vivo X3L Screen Resolution: 1920 × 1080 System Language: Chinese	1. Click the Lucky Draw button for three seconds 2. Click the Stop Draw button	The turntable stops rotating successfully, and the lottery is successful.	
Report4	Operating System: Android 4.3 Phone model: Vivo X3L Screen Resolution: 1920 × 1080 System Language: Chinese	1. Modify personal profile 2. Modify background image or personal profile 3. Click save	Prompt "There is an unknown error", and the modification fails to save.	
Report5	Operating System: Android 4.3 Phone model: Vivo X3L Screen Resolution: 1920 × 1080 System Language: Chinese	1. View Comments 2. Enter Comments 3. Click Submit	Can't quickly find out comments that have been posted.	

The information-retrieval-based algorithm uses each test case's execution information or source code to construct a corresponding document collection for each test case. The machine-learning-based algorithm uses machine learning technology, which builds a model from sample input to make predictions on new data.

III. METHODOLOGY

In this section, we introduce our crowdsourced test report prioritization method in detail. Figure 2 shows the overall framework of our test report prioritization.

A. PREPROCESSING

After collecting the crowdsourced test reports, we first preprocess the report. These test reports strictly include four parts: Environment, Input, Description, and Screenshot. Our preprocessing operations for the report include three processes: word segmentation, stop word removal and synonym replacement.

1) WORD SEGMENTATION

Word segmentation is a natural language processing (NLP) task. There are many effective word segmentation tools for

different languages [12]. Since our report is composed in Chinese, we use the Jieba library to segment sentences into words. Jieba is a word segmentation algorithm specially used for Chinese word segmentation. It uses a prefix dictionary to achieve efficient word graph scanning, generates a directed acyclic graph composed of all possible generations of Chinese characters in the sentence, then uses dynamic programming to find the maximum probability path, and finds the maximum cut based on word frequency. For unregistered words, the HMM (Hidden Markov Model) model [13] based on the ability to form Chinese characters is used, and the Viterbi algorithm is used.

2) STOP WORD REMOVAL

In crowdsourced testing, workers usually come from different workplaces, and they have different language preferences and expression habits. Therefore, the test report will contain some meaningless words, and these words do not need to be part of the features of the test report. We filter out these useless words according to the stop word list, usually called “stop words” in the NLP literature.

3) SYNONYM REPLACEMENT

In crowdsourced testing, workers may use different words to express the same concept in the test report. For example, “turn on” and “open” refer to the same semantics. To alleviate this problem, we perform the synonym replacement operation. We adopt the synonym library of Language Technology Platform (LTP) [14], which is considered to be one of the best Chinese NLP platforms.

B. TEXT FEATURE EXTRACTION

We extract three types of report text features, i.e., TF-IDF (Term Frequency and Inverse Document Frequency) feature, word embedding feature, and one-hot encoding feature.

1) TF-IDF FEATURE

TF-IDF is a statistical method used to evaluate the importance of a word to a document set or a document in a corpus. The main idea of TF-IDF is that if a word or phrase appears in an article with a high frequency of TF and rarely appears in other articles, it is considered that the word or phrase has good classification ability and is suitable for classification. Specifically, given a term T and a report R , $TF(T, R)$ is the frequency of the term T in the report R . $IDF(T)$ is obtained by dividing the total number of reports by the number of reports containing the term T , and then taking the logarithm of the obtained quotient. The calculation formula of TF-IDF is:

$$TF - IDF = TF(T, R) \times IDF(T) \quad (1)$$

Through the above formula, the text description of report R can be expressed as a TF-IDF vector, i.e., $R = (W_1, W_2, \dots, W_i)$, where W_i represents the TF-IDF value of the i -th item in the report R .

2) WORD EMBEDDING FEATURE

Word embedding is a feature learning technology in natural language processing. A single word is represented as an absolute number vector in a predefined vector space, and each word is mapped to a vector [5]. We use the Word2Vec model to obtain the word embedding of a report. Word2Vec is one of the methods of word embedding. The trained word embedding model will convert each word into a 100-dimensional vector. A crowdsourced test report contains multiple words, which can be converted into a matrix. Each row of the matrix represents a term in the report. Then, we convert the report matrix into a vector by averaging all word vectors contained in the report.

Specifically, given a reporting matrix with a total of M rows, the i -th row of the matrix is denoted as r_i , and the converted report vector R_w is generated as follows:

$$R_w = \frac{\sum_i r_i}{M} \quad (2)$$

According to the above formula, each crowdsourced test report can be expressed as a word embedding vector.

3) ONE-HOT ENCODING FEATURE

One-hot encoding is also known as one-bit effective encoding. One-hot encoding is a word represented by a vector of length V , where only one position is 1, and the rest are 0. V is the size of the lexicon in the corpus. We construct a corpus based on the collected crowdsourced test reports and convert each report into a report vector encoded by 0 and 1.

C. PRIORITIZATION STRATEGY

In this section, we use four prioritization strategies to prioritize test reports. These four strategies are total greedy algorithm, additional greedy algorithm, genetic algorithm, and ART.

1) TOTAL GREEDY ALGORITHM

The total greedy algorithm is widely used to solve the test case prioritization problem, which focuses on always selecting the current best test case during the test case prioritization period. The goal of the total greedy algorithm is to select tests that contain more sentences and prioritize the test cases according to the descending order of the sentences covered by each test case. The total greedy algorithm directly calculates the program entity coverage of each test case and sorts it from high to low. Assuming that there are m test cases and n program entities that need to be covered, the time complexity of this strategy is $O(mn)$. For this strategy, we use the one-hot encoding feature to prioritize reports.

2) ADDITIONAL GREEDY ALGORITHM

The additional greedy algorithm introduces a feedback mechanism. This algorithm believes that when the executed test cases cover some program entities, the remaining test cases do not need to be considered for coverage of the above

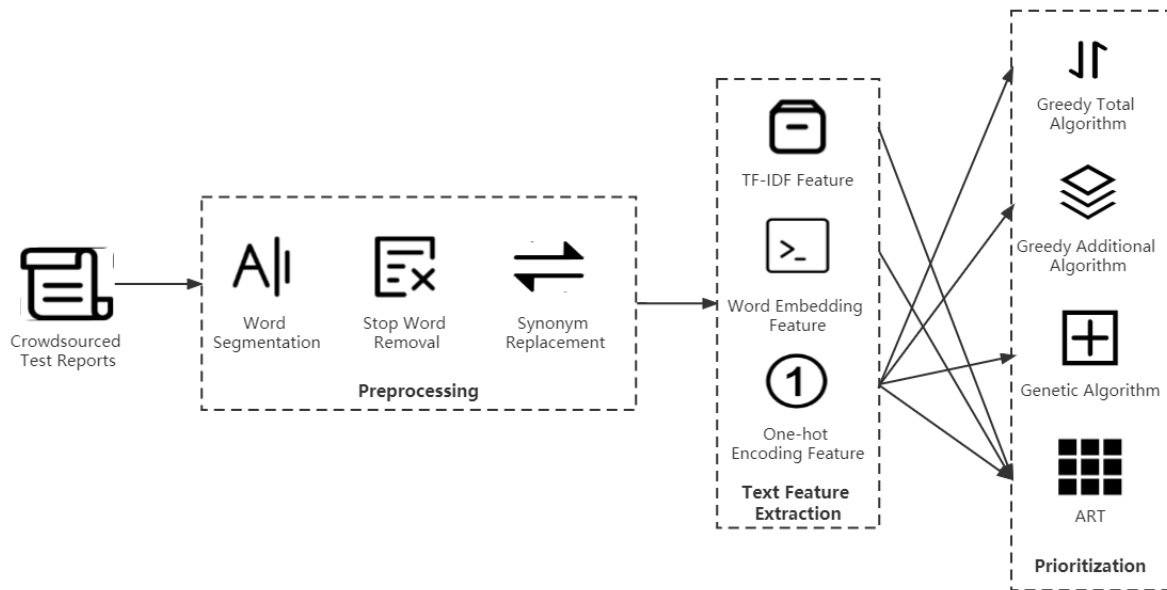


FIGURE 2. The overall framework of test report prioritization.

program entities. Specifically, each time a test case is executed, the coverage information of the remaining test cases needs to be updated in real-time. When all program entities are covered, reset these program entities to uncovered and iteratively apply the above process to the remaining test cases. The time complexity of the strategy is $O(m^2n)$. This algorithm uses the one-hot encoding feature to prioritize reports.

The greedy method is a standard method to solve the test case prioritization problem. After collecting the historical coverage information of test cases, according to the coverage ability of each test case's program entities (such as statements, branches, or functions), set the weight for it, and apply the greedy method to guide the ordering of test cases. The assumption is that improving the early coverage of program entities helps improve the early detection rate of defects. The total greedy and additional greedy algorithms focus on searching for the optimal local solution for prioritization, so the prioritization result may not be optimal.

3) GENETIC ALGORITHM

The genetic algorithm is an evolutionary algorithm. Its basic principle is to imitate the evolutionary law of "natural selection in competition and survival of the fittest" in Darwin's biological evolution theory. The genetic algorithm encodes the problem parameters into chromosomes. It then uses iterative methods to perform selection, crossover, and mutation to exchange information about chromosomes in the population and finally evolve into chromosomes that meet the optimization goals. In this algorithm, each test sequence is encoded in an n -size array representing an instance of a chromosome. A set of test sequences is randomly generated as the initial individual in the initial step. Combine the selected individuals guided by the fitness function to generate a new generation.

This process is iterative. The entire search process stops until the requirements are met. The feature of this algorithm for prioritization is a one-hot encoding feature.

The genetic algorithm belongs to search-based prioritization, characterized by finding the optimal solution under the guidance of a predefined fitness function in the search space.

4) ART

ART was proposed by Jiang *et al.* in 2009. Its primary purpose is to prioritize a given set of test cases. It iteratively builds a set of candidate test cases and then selects a test case from the candidate set until all the given cases are selected. It calls a generated process to build the candidate set. The generation process iteratively constructs a set of unselected test cases by randomly adding the remaining test cases to the candidate set, as long as they can increase the program's coverage and the candidate set is not complete yet. Another procedure selection function needs to be called to decide which candidate test case to choose. The latter requires a function $f1$ to calculate the distance between a pair of test cases and another function $f2$ to return the index of the selected test case furthest from the priority set. The function $f1$ uses the Jaccard distance between two test cases based on the coverage structure. For a test suite with m test cases and n sentences, the time complexity of the algorithm is $O(m^2)$ in the best case and $O(m^3n)$ in the worst case. We use the TF-IDF feature, word embedding feature, and one-hot encoding feature for the ART algorithm.

IV. EXPERIMENT

In this section, we evaluate the effectiveness of four test report prioritization algorithms. We introduce the experimental settings and the baseline method compared with the four

TABLE 2. Experiment application.

App	App Category	Reports
A1	Education	338
A2	Finance	145
A3	Music	41
A4	Reading	39
A5	Life	160

prioritization algorithms. We also set up two research questions (RQ). Finally, we introduce the evaluation metric.

A. EXPERIMENT SETUP

To evaluate the four prioritization algorithms, we design an empirical experiment. We collect 723 crowdsourced test reports from 5 different mobile applications to complete the experiment. Table 2 shows the detailed information. The labels of these applications range from A1 to A5, and the number of test reports for different applications ranges from 39 to 338.

B. BASELINE

We select two baselines for comparison to verify the effectiveness of four prioritization methods. The first baseline is the **Random** strategy, which is widely used in software testing. **Random** strategy randomly selects a test report for sorting, forming a priority ranking list. The second baseline is the **Best** strategy, which is an ideal prioritization technique. To compare these prioritization methods fairly, we repeat the experiment 10 times to collect experimental data.

C. RESEARCH QUESTION

In the experiment, we design the following two research questions.

- RQ1: How effective are these four prioritization algorithms?
- RQ2: How far are these four methods from the **Best** strategy?

When the priority method is not applied, testers will apply the **Random** strategy to prioritize the test reports. Only when the prioritization method can exceed the **Random** strategy will testers use it. RQ1 evaluates the effectiveness of four prioritization methods.

In practice, it is difficult to design effective methods in all situations. Therefore, it is valuable to understand the gap between the current four methods and the theoretically **Best** method. RQ2 evaluates the room for improvement in the four prioritization methods.

D. EVALUATION METRIC

In order to measure the effectiveness of four prioritization methods, we adopt APFD (Average Percentage of Failure Detection) [7]. Test report prioritization simulates the ideal test case prioritization, and APFD is an evaluation metric that is widely used in test case prioritization [15]. Therefore, we use APFD to evaluate the effectiveness of four test report

prioritization methods. We will record the index of the first test report that showed it for each bug. According to the order of the test reports and which test reports show which fault information, we can calculate the APFD value to measure the effectiveness of the prioritization method. The value of APFD ranges from 0 to 1. The higher the APFD value, the better the prioritization result. The calculation formula of APFD is as follows,

$$APFD = 1 - \frac{\sum_{i=1}^M T_{fi}}{n \times M} + \frac{1}{2 \times n} \quad (3)$$

where n is the number of test reports, M is the total number of bugs shown in the test report, and T_{fi} is the index of the first test report showing the bug i .

V. RESULT ANALYSIS

In this section, we analyze the experimental results to answer RQ1 and RQ2. The results of all prioritization methods are shown in Figure 3. Figure 3 shows a box plot of APFD results for ten experimental runs of five applications. The application label is displayed on the horizontal axis, and the APFD value is displayed on the vertical axis.

A. ANSWER TO RQ1

RQ1: How effective are these four prioritization algorithms?

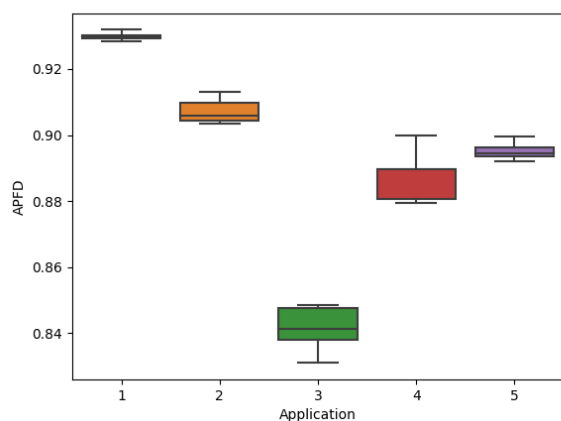
According to the results shown in Figure 3, we can find that these four priority ranking methods are better than **Random**. For the greedy strategy, the average APFD calculated by the additional algorithm is slightly better than the total on these five applications. There is little difference in the results obtained by using three different features to prioritize the ART strategy. On the two applications, A2 and A5, the calculated average APFD value is the same. In the remaining three applications, the calculated average APFD values are within 3%. In these three strategies, the average APFD value of the genetic algorithm in each application is above 0.9. In addition, the box plot and Table 3 show that these four methods are more stable than the random method. In summary, we found that these four prioritization methods can improve the effectiveness of the prioritization of test reports.

B. ANSWER TO RQ2

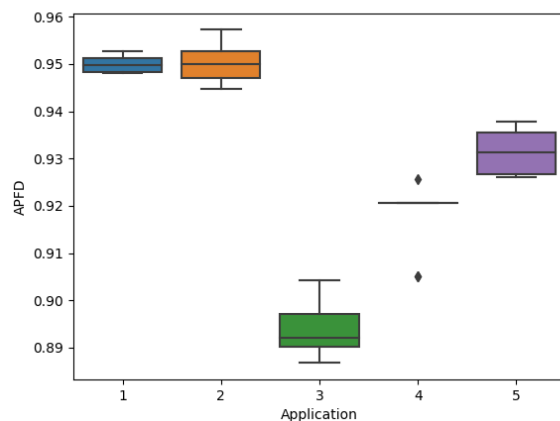
RQ2: How far are these four methods from the **Best** strategy?

In order to avoid the contingency of the experimental results, we run the random method ten times on the test report data set of each application and calculate the average APFD. Table 4 to 9 shows the comparison results of the four prioritization strategies and the baseline method. In all the tables, "Improvement" means the improvement of the method compared to **Random**. The calculation formula is $\frac{X - \text{Random}}{\text{Random}}$, where X is our method. "Gap" means the gap between the method and the **Best**. The calculation formula is $\frac{\text{Best} - X}{X}$, where X is our method.

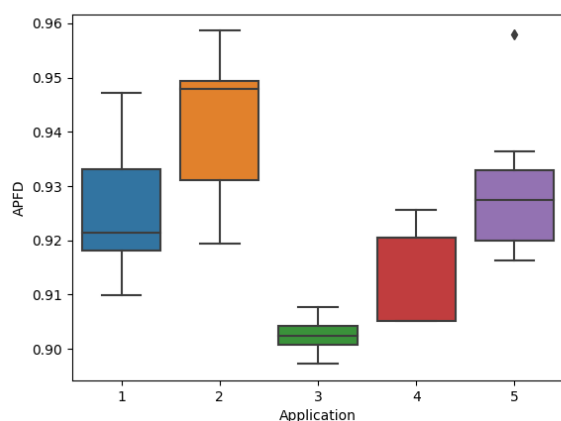
As can be seen from the table, we observe that the results of these four strategies are better than random and close to the best results. Among the two greedy strategies



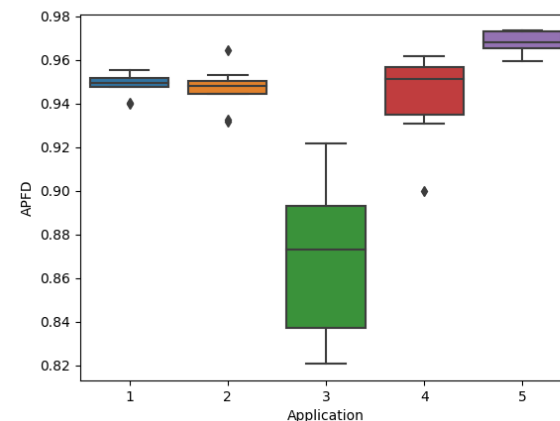
(a) Total Greedy



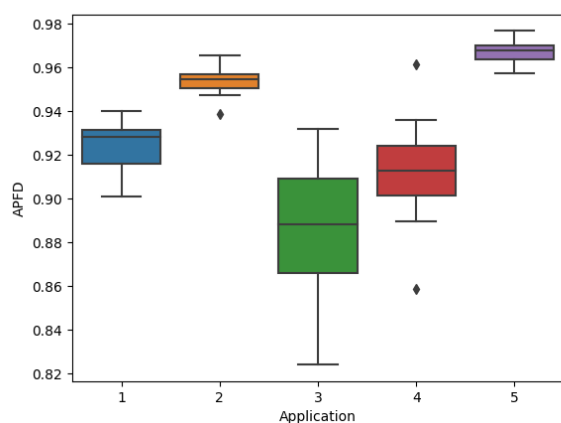
(b) Additional Greedy



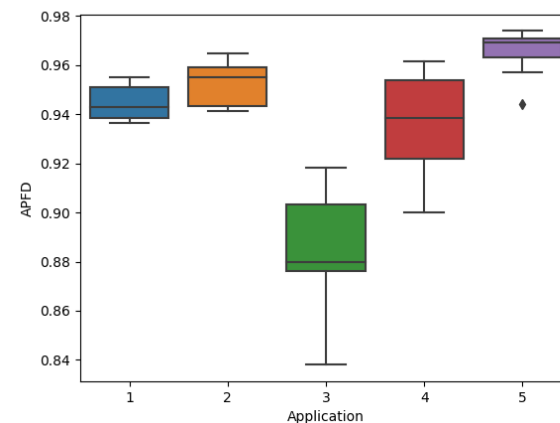
(c) Genetic



(d) ART(TF-IDF)



(e) ART(Word Embedding)



(f) ART(One-hot Encoding)

FIGURE 3. Experimental results.

algorithms, the additional greedy algorithm performs better than the total greedy algorithm in these five applications, and the gap with Best is smaller. For the ART algorithm,

we used three features. It can be seen from the results that the APFD calculated using these three features is not very different. Overall, the APFDs calculated by the four strategies

TABLE 3. Baseline.

Application	Best	Random
A1	0.9512	0.9119
A2	0.9526	0.9321
A3	0.9098	0.8551
A4	0.9513	0.8841
A5	0.9665	0.9357

TABLE 4. Total greedy.

Application	Average APFD	Improvement	Gap
A1	0.93	1.97%	2.30%
A2	0.91	0.67%	5.02%
A3	0.84	1.63%	8.17%
A4	0.89	0.41%	7.17%
A5	0.90	4.34%	7.98%

TABLE 5. Additional greedy.

Application	Average APFD	Improvement	Gap
A1	0.95	4.17%	0.13%
A2	0.95	1.93%	0.26%
A3	0.89	4.52%	1.80%
A4	0.91	3.83%	3.63%
A5	0.93	5.79%	3.79%

TABLE 6. Genetic.

Application	Average APFD	Improvement	Gap
A1	0.92	1.43%	2.84%
A2	0.94	1.05%	1.10%
A3	0.90	5.50%	0.85%
A4	0.91	3.48%	3.98%
A5	0.93	1.73%	3.35%

TABLE 7. ART(One-hot encoding).

Application	Average APFD	Improvement	Gap
A1	0.95	3.61%	0.67%
A2	0.95	2.18%	0.01%
A3	0.86	3.59%	2.65%
A4	0.94	5.86%	1.62%
A5	0.97	3.21%	0.08%

are similar, and all are better than randomly ordered test reports.

C. THREATS TO VALIDITY

In our research, there are some general threats to its effectiveness.

1) CROWDSOURCED WORKER CAPACITY

The capacity of the crowdsourced workers is not controlled, so there may be some low-quality reports. However, these four strategies identified the errors described in the reports even when some reports were of low quality. If the test report

TABLE 8. ART(TF-IDF).

Application	Average APFD	Improvement	Gap
A1	0.95	4.01%	0.29%
A2	0.95	1.55%	0.64%
A3	0.87	1.51%	4.82%
A4	0.94	6.79%	0.76%
A5	0.97	3.48%	0.18%

TABLE 9. ART(Word embedding).

Application	Average APFD	Improvement	Gap
A1	0.92	1.34%	2.93%
A2	0.95	2.34%	0.14%
A3	0.88	3.42%	2.88%
A4	0.91	3.19%	4.72%
A5	0.97	3.37%	0.07%

contains no errors, these policies will classify the report into a single category and will not affect the priority of other reports.

2) EXPERIMENTAL SOFTWARE SELECTION

We selected five applications widely used on the Internet that are not specifically designed for our research. Due to time and cost constraints, we only collected crowdsourced test reports for these five applications. The amount and type of software may threaten the generalization of our conclusions. However, there are different types of software in our choice, which may reduce the threat to a certain extent.

3) TEST REPORT TEXT

In our experiment, all test reports are written in Chinese, which may threaten to apply these methods to other natural language test reports. However, the current natural language processing technology is very mature, and there are different toolkits for processing different languages. Moreover, our preprocessing operations do not limit the type of language. In summary, we adopt different toolkits for different languages to deal with them, which can reduce this threat.

VI. RELATED WORK

In this section, we introduce the work related to our research, including crowdsourced testing, crowdsourced test report processing, and test case prioritization.

A. CROWDSOURCED TESTING

Crowdsourcing refers to a company or organization that outsources work tasks performed by employees in the past to non-specific and usually large mass networks in a free and voluntary manner. Since crowdsourcing can simulate different conditions of use and the economic cost is relatively low [1], [16], crowdsourcing has been applied in many fields.

Crowdsourced testing is a relatively new trend in software engineering [17]. Liu *et al.* [18] studied the results of applying crowdsourced testing to usability testing and also examined empirical comparisons between crowdsourced

usability testing and more traditional face-to-face usability testing results. Through experiments, they found that crowdsourcing exhibits some limitations compared to traditional lab environments, but its applicability and value in usability testing are clear. In addition to applying crowdsourced testing to usability testing, Dolstra *et al.* applied crowdsourced testing to GUI testing by instantiating a virtual machine running the system under test and giving testers access to the virtual machine through their web browsers. Outsourced to a large number of workers from the Internet [19]. Nebeling *et al.* [20] developed a toolkit for crowdsourced testing of web interfaces that not only efficiently recruits large numbers of test users but also evaluates websites under many different conditions.

Another part of the research focuses on the selection of crowdsourced workers. Not all workers are qualified to perform a particular test task. Cui *et al.* [31] proposed a multi-objective population worker selection method (MOOSE), which utilizes the widely used multi-objective evolutionary algorithm NSGA-II to optimize three objectives when selecting workers. ExReDiv [32], a novel hybrid method consisting of an empirical strategy, a correlation strategy, and a diversity strategy, is used to select a set of workers for testing tasks. Furthermore, Xie *et al.* [33] propose a new research problem called Crowdsourced Test Quality Maximization under Context Coverage Constraints.

The above researches apply crowdsourced testing methods to traditional software testing activities to solve some problems in traditional software testing. Instead, we propose to migrate the test case prioritization method to test report prioritization to solve the problem in crowdsourced testing.

B. CROWDSOURCED TEST REPORT PROCESSING

In crowdsourced testing, a large number of test reports will be generated, which requires developers to spend a lot of time checking, and the key issue is to improve the efficiency of developers reviewing test reports. There has been extensive research on handling crowdsourced test reports to better assist developers in reviewing reports.

Wang *et al.* [21] proposed a clustering-based classification method that first clusters similar reports together and then uses an ensemble method to build a classifier based on the most similar clusters, which can reduce the effort required for manual inspection and facilitate project management in crowdsourced testing. This is the first work to address the classification of test reports in practical industrial crowdsourcing testing practice. Wang *et al.* [22] proposed Local Based Active Classification (LOAF) to classify accurate faults in crowdsourced test reports. LOAF recommends a small set of the most informative instances in the local neighborhood, asks the user their labels, and then learns a classifier based on the local neighborhood. Furthermore, Chen *et al.* proposed a new framework named Test Report Augmentation Framework (TRAF) to improve the quality of inspection test reports [23]. They enhance test reports with additional

useful information in repeated test reports, which can help developers better understand and fix bugs. Sun *et al.* [24] used an information retrieval model to more accurately detect duplicate bug reports.

The above methods include report classification and duplicate detection, all of which select part of the test report to represent all the test reports. In addition to this, some studies have attempted to assist in the generation of test reports. Liu *et al.* used the spatial pyramid matching technique to measure similarity and extract features from screenshot images to generate descriptions for screenshots, which could allow developers to understand mobile crowdsourced test reports better [25]. Yu generate crowdsourced bug reports by analyzing crowdsourced uploaded error screenshots using image understanding techniques to provide operational guidance for crowdsourcing [26].

C. TEST CASE PRIORITIZATION

Test case prioritization is proposed in regression testing to handle the execution order of test cases by scheduling to detect more failures earlier [27]. Rothermel *et al.* [28] proposed various basic test case prioritization techniques, including total techniques and additional techniques. Hao *et al.* [29], [30] proposed a unified priority model that uses a probabilistic model to bridge the gap between total and additional algorithms.

Li *et al.* [34] applied a meta-heuristic search-based algorithm to test case prioritization, and they applied the steepest ascent hill-climbing algorithm and genetic algorithm. In addition, Zhang *et al.* [35] first applied Integrated linear programming to solve the time-aware test case prioritization problem. Nguyen *et al.* [36] proposed an information retrieval-based method to prioritize test cases for web services, which used an identifier document extracted from the execution trace to represent each test case and a web service change description as input queries for information retrieval. Tonella *et al.* [37] proposed a test case prioritization method based on machine learning, using the priority indicators defined by user use cases, as well as test case information such as coverage and failure propensity indicators as features, to train a model to predict test cases priority. Ledru *et al.* [15] used string distance comparisons on test case texts to prioritize test cases and provided empirical results on different distances. In addition to this, [38] proposed ESBS, which iteratively selects test cases from each cluster and uses spectral information in cluster test selection.

Test case prioritization techniques have matured, which inspired us to migrate test case prioritization techniques into test report prioritization. Most test case prioritization techniques use execution profiles, whereas we use test reports in natural language. Test case prioritization enables more failures to be detected earlier by executing test cases. Prioritizing test reports can improve tester review efficiency by checking test reports.

VII. CONCLUSION

We employ a diversity strategy to prioritize test reports to help developers prioritize and filter useful report information. We process the text in the test reports using NLP techniques and use three different features to represent each report. These three features are the one-hot encoding, TF-IDF, and word embedding. After obtaining the feature vector, we use the total greedy algorithm, additional greedy algorithm, genetic algorithm, and ART to prioritize the test report. Five mobile crowdsourced test report datasets are used to evaluate the effectiveness of the test report prioritization method. The results of the empirical study show that these four prioritization methods all have good performance and can effectively help developers reduce inspection costs.

In the future, we will continue to dig into the adaptive test report prioritization methods, not only by borrowing techniques for prioritizing test cases. In addition, we will also try to design a new test report prioritization technology, starting from the test report itself, analyzing the images and text information contained in the test report, and extracting the corresponding features for report prioritization. And we will apply these techniques to actual production to solve test report-related tasks.

REFERENCES

- [1] K. Mao, L. Capra, M. Harman, and Y. Jia, "A survey of the use of crowd-sourcing in software engineering," *J. Syst. Softw.*, vol. 126, pp. 57–84, Apr. 2017.
- [2] W. Wu, W.-T. Tsai, and W. Li, "An evaluation framework for software crowdsourcing," *Frontiers Comput. Sci.*, vol. 7, no. 5, pp. 694–709, Oct. 2013.
- [3] Y. Lou, J. Chen, L. Zhang, and D. Hao, "A survey on regression test-case prioritization," in *Advances in Computers*, vol. 113. Amsterdam, The Netherlands: Elsevier, 2019, pp. 1–46.
- [4] B. Jiang, Z. Zhang, W. K. Chan, and T. H. Tse, "Adaptive random test case prioritization," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, Nov. 2009, pp. 233–244, doi: 10.1109/ASE.2009.77.
- [5] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 3111–3119.
- [6] G. Salton and C. T. Yu, "On the construction of effective vocabularies for information retrieval," *ACM SIGPLAN Notices*, vol. 10, no. 1, pp. 48–60, 1975.
- [7] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Trans. Softw. Eng.*, vol. 27, no. 10, pp. 929–948, Oct. 2001, doi: 10.1109/32.962562.
- [8] J. Howe, "The rise of crowdsourcing," *Wired Mag.*, vol. 14, no. 6, pp. 1–4, Jun. 2006.
- [9] E. Estellés-Arolas and F. González-Ladrón-De-Guevara, "Towards an integrated crowdsourcing definition," *J. Inf. Sci.*, vol. 38, no. 2, pp. 189–200, Apr. 2012.
- [10] X. Zhang, Y. Feng, D. Liu, Z. Chen, and B. Xu, "Research progress of crowdsourced software testing," *J. Softw.*, vol. 29, no. 1, pp. 69–88, 2018.
- [11] J. A. Jones and M. J. Harrold, "Test-suite reduction and prioritization for modified condition/decision coverage," *IEEE Trans. Softw. Eng.*, vol. 29, no. 3, pp. 195–209, Mar. 2003.
- [12] S. Foo and H. Li, "Chinese word segmentation and its effect on information retrieval," *Inf. Process. Manage.*, vol. 40, no. 1, pp. 161–190, Jan. 2004.
- [13] L. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proc. IEEE*, vol. 77, no. 2, pp. 257–286, Feb. 1989.
- [14] W. Che, Z. Li, and T. Liu, "LTP: A Chinese language technology platform," in *Proc. Coling, Demonstrations*, 2010, pp. 13–16.
- [15] Y. Ledru, A. Petrenko, and S. Boroday, "Using string distances for test case prioritisation," in *Proc. IEEE/ACM Int. Conf. Automated Softw. Eng.*, Nov. 2009, pp. 510–514.
- [16] R. Gao, Y. Wang, Y. Feng, Z. Chen, and W. E. Wong, "Successes, challenges, and rethinking—An industrial investigation on crowdsourced mobile application testing," *Empirical Softw. Eng.*, vol. 24, no. 2, pp. 537–561, Apr. 2019.
- [17] S. Alyahya, "Crowdsourced software testing: A systematic literature review," *Inf. Softw. Technol.*, vol. 127, Nov. 2020, Art. no. 106363.
- [18] D. Liu, R. G. Bias, M. Lease, and R. Kuipers, "Crowdsourcing for usability testing," *Proc. Amer. Soc. Inf. Sci. Technol.*, vol. 49, no. 1, pp. 1–10, 2012.
- [19] E. Dolstra, R. Vliegendorst, and J. Pouwelse, "Crowdsourcing GUI tests," in *Proc. IEEE 6th Int. Conf. Softw. Test., Verification Validation*, Mar. 2013, pp. 332–341, doi: 10.1109/ICST.2013.44.
- [20] M. Nebeling, M. Speicher, M. Grossniklaus, and M. C. Norrie, "Crowd-sourced web site evaluation with crowdstudy," in *Proc. Int. Conf. Web Eng.*, Berlin, Germany: Springer, 2012, pp. 494–497.
- [21] J. Wang, Q. Cui, Q. Wang, and S. Wang, "Towards effectively test report classification to assist crowdsourced testing," in *Proc. 10th ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas.*, Sep. 2016, pp. 1–10.
- [22] J. Wang, S. Wang, Q. Cui, and Q. Wang, "Local-based active classification of test report to assist crowdsourced testing," in *Proc. 31st IEEE/ACM Int. Conf. Automated Softw. Eng.*, Aug. 2016, pp. 190–201.
- [23] X. Chen, H. Jiang, Z. Chen, T. He, and L. Nie, "Automatic test report augmentation to assist crowdsourced testing," *Frontiers Comput. Sci.*, vol. 13, no. 5, pp. 943–959, Oct. 2019.
- [24] C. Sun, D. Lo, X. Wang, J. Jiang, and S.-C. Khoo, "A discriminative model approach for accurate duplicate bug report retrieval," in *Proc. 32nd ACM/IEEE Int. Conf. Softw. Eng. (ICSE)*, 2010, pp. 45–54.
- [25] D. Liu, X. Zhang, Y. Feng, and J. A. Jones, "Generating descriptions for screenshots to assist crowdsourced testing," in *Proc. IEEE 25th Int. Conf. Softw. Anal., Evol. Reeng. (SANER)*, Mar. 2018, pp. 492–496.
- [26] S. Yu, "Crowdsourced report generation via bug screenshot understanding," in *Proc. 34th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2019, pp. 1277–1279.
- [27] W. E. Wong, J. R. Horgan, S. London, and H. Agrawal, "A study of effective regression testing in practice," in *Proc. 8th Int. Symp. Softw. Rel. Eng.*, 1997, pp. 264–274.
- [28] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Test case prioritization: An empirical study," in *Proc. IEEE Int. Conf. Softw. Maintenance*, Aug. 1999, pp. 179–188.
- [29] D. Hao, L. Zhang, L. Zhang, G. Rothermel, and H. Mei, "A unified test case prioritization approach," *ACM Trans. Softw. Eng. Methodol.*, vol. 24, no. 2, pp. 1–31, Dec. 2014.
- [30] L. Zhang, D. Hao, L. Zhang, G. Rothermel, and H. Mei, "Bridging the gap between the total and additional test-case prioritization strategies," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, May 2013, pp. 192–201.
- [31] Q. Cui, S. Wang, J. Wang, Y. Hu, Q. Wang, and M. Li, "Multi-objective crowd worker selection in crowdsourced testing," in *Proc. SEKE*, vol. 17, 2017, pp. 218–223.
- [32] Q. Cui, J. Wang, G. Yang, M. Xie, Q. Wang, and M. Li, "Who should be selected to perform a task in crowdsourced testing?" in *Proc. IEEE 41st Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, Jul. 2017, pp. 75–84.
- [33] M. Xie, Q. Wang, G. Yang, and M. Li, "COCOON: Crowdsourced testing quality maximization under context coverage constraint," in *Proc. IEEE 28th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Oct. 2017, pp. 316–327.
- [34] Z. Li, M. Harman, and R. M. Hierons, "Search algorithms for regression test case prioritization," *IEEE Trans. Softw. Eng.*, vol. 33, no. 4, pp. 225–237, Apr. 2007.
- [35] L. Zhang, S.-S. Hou, C. Guo, T. Xie, and H. Mei, "Time-aware test-case prioritization using integer linear programming," in *Proc. 18th Int. Symp. Softw. Test. Anal. (ISSTA)*, 2009, pp. 213–224.
- [36] C. D. Nguyen, A. Marchetto, and P. Tonella, "Test case prioritization for audit testing of evolving web services using information retrieval techniques," in *Proc. IEEE Int. Conf. Web Services*, Jul. 2011, pp. 636–643.
- [37] P. Tonella, P. Avesani, and A. Susi, "Using the case-based ranking methodology for test case prioritization," in *Proc. 22nd IEEE Int. Conf. Softw. Maintenance*, Sep. 2006, pp. 123–133.
- [38] S. Yan, Z. Chen, Z. Zhao, C. Zhang, and Y. Zhou, "A dynamic test cluster sampling strategy by leveraging execution spectra information," in *Proc. 3rd Int. Conf. Softw. Test., Verification Validation*, 2010, pp. 147–154.



PENGHUA ZHU was born in Handan, Hebei, China, in 1980. He is currently working with the North China Institute of Aerospace Engineering and the Principal of the School of Computer. His title is a Senior Experimenter. His research interests include software engineering and computer application technology.



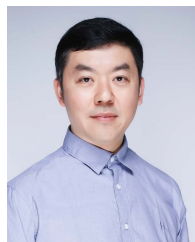
HUIMIN REN was born in Hohhot, Inner Mongolia, China, in 1981. She is currently working as a Senior Engineer with the Beijing Aerospace Automatic Control Institute. She is also the Deputy Director of the System Software Engineering Research Department, Beijing Aerospace Automatic Control Institute. Her research interests include software engineering and computer application technology.



YING LI was born in Langfang, Hebei, China, in 1982. Her title is an Associate Professor. Her research interests include software testing, software quality assurance, and computer application technology.



TONGYU LI received the B.E. degree in computer science and technology from Northwestern Polytechnical University, in 2016. She is currently pursuing the master's degree in software engineering with Nanjing University.



XIAOLEI SUN received the bachelor's degree in automation and the master's degree in control theory and control engineering from the Hefei University of Technology. He is currently a member of the Party Committee and the Vice President of the Wuhu Vocational and Technical College. He has published more than 20 high-level academic papers and authorized six invention patents. His research interests include electronic information and vocational education.

...